BIASHARA MALL

Build Log

A technical record of what was built, what broke, and why each decision was made. Written as we go, newest entry last. **Why this file exists:** in six weeks nobody remembers why the config normalises a database driver, or why the test suite refuses to run without a second database. The code says *what*; this says *why it had to be that way*, including the dead ends — a fix whose reasoning is lost gets undone by the next person who finds it inconvenient.

2026-08-05 → 08 · Re-baseline and P0 Foundation

Context: the project changed shape twice in one day

The session opened intending to build a 14-day TypeScript plan. Two discoveries changed that.

1. A working Python implementation already existed. `Project TIKTOK` (`Data-Amigo/PROJECT_TIKTOK`) — 63 commits, 110 passing tests, live on Railway. It already covered most of what the plan called Days 1–6: Apify ingestion, a Gemini vision draft agent, an M-Pesa Daraja client with an idempotent callback, auth, and a public storefront.

2. WhatsApp became the interface, not the exit. Meta business verification is in progress, which reverses TIKTOK's 2026-07-25 pivot — that pivot had moved the agentic close onto the web *specifically because* Meta was blocking it.

Decisions taken:

- **Python, not TypeScript.** Fredrick is materially stronger in Python and can debug it unaided. On a solo build that outweighs framework preference. Pydantic replaces Zod and does double duty as the LLM structured-output schema.
- **Build fresh; TIKTOK is a reference, not a foundation.** Its proven modules are read and adapted, never copied blind and never reinvented either.
- **Three pillars**: Soko Commerce, Soko Intel, Soko AI.
- **The mini app is the WhatsApp in-app browser, not WhatsApp Flows.** Removes the Flows approval dependency, gives full UI control, and debugs in an ordinary browser. It also collapses two surfaces into one — the same server-rendered pages are the in-app storefront *and* the SEO presence.

A finding that was already known

Mid-session, analysis of the 24 exported POC captions showed **zero mention KSh** and only three containing any price-like number — so prices cannot be parsed from captions.

This was presented as a discovery. It was not: TIKTOK's `spike_00` established it on 2026-07-22, and `agent/draft.py` already implements the full cascade (cover image, then Gemini watching *and listening* to the clip), proven live at KES 500/550/550/600.

Lesson recorded deliberately: read the reference project's docs *before* analysing its data. Several hours went into re-deriving a conclusion that was already written down in `docs/CONCEPTS.md`.

Documents produced

Generated rather than hand-written, from `docs/src/`, so a plan change is an edit and a re-run instead of four documents silently drifting apart:

OUTPUT	SOURCE	HOW
`SokoLink_Business_Document.pdf`	`src/business.html`	headless Chrome `--print-to-pdf`
`SokoLink_Technical_Document.pdf`	`src/technical.html`	headless Chrome `--print-to-pdf`
`SokoLink_Project_Flow.pptx` (13 slides)	`src/build_deck.py`	python-pptx

The deck was exported to PNG via PowerPoint COM and inspected slide by slide — which caught two real layout collisions (callout boxes overlapping the footer on the tech-stack and hard-parts slides). Generating a deck without looking at it is not verification.

P0 — Foundation

Retired the TypeScript scaffold. `apps/`, `packages/`, `node_modules` (704 MB), and all the JS tooling config. Uncommitted, so nothing was lost from history.

Built the FastAPI spine:

```
backend/
  app/
    main.py       thin — wires routers, owns nothing else
    config.py     typed settings; NOTHING else reads os.environ
    db.py         engine + session; pool_pre_ping for cloud Postgres
    api/health.py liveness + readiness
    models/ schemas/ services/   layered, documented, empty until P1
  alembic/        migrations, wired to app config
  tests/          pytest with transactional fixtures
  scripts/        setup utilities
```

Design decisions worth keeping:

- **`/health` and `/health/ready` are separate endpoints.** They answer different questions with different consequences. Liveness failing means "restart me"; readiness failing means "stop routing traffic, but do *not* restart — the database is down and restarting will not fix that." Collapsing them makes a slow database into a platform restart-loop. A test asserts liveness never opens a database connection.
- **`pool_pre_ping=True` and `pool_recycle=300`.** Cloud Postgres silently drops idle connections. Without pre-ping the app hands a request a dead connection and the user sees a 500 that vanishes on retry — the worst kind of bug to chase.
- **Settings are lazy (`@lru_cache`), not evaluated at import.** An eager `settings = Settings()` throws the moment anything imports the module, which breaks tests and tooling that only need to resolve it.
- **`extra="ignore"` on Settings.** Railway injects `PORT` and `RAILWAY_*`. Without this, deploys fail on someone else's config.
- **Alembic reads the URL from `app.config`, not `alembic.ini`.** The URL is a secret and `alembic.ini` is committed. One source means the app, the tests and the migrations can never disagree about which database they mean.

Five things that broke, and the fixes

1. ruff reported 42,462 errors. It was linting `.venv`. Added it to `exclude`. Real error count: 1.

2. `alembic/env.py` failed import sorting. An explanatory comment sat *between* imports, splitting the block. Moved the comment below the imports — where it now also warns that the apparently-unused `Base` import is load-bearing: without it, autogenerate sees an empty schema and writes a migration that drops every table.

3. `ModuleNotFoundError: No module named 'psycopg2'`. SQLAlchemy reads a bare `postgresql://` URL as "use psycopg2", which is not installed — we use psycopg 3. Railway hands out exactly that bare URL.

Fixed in `config.py` with `_with_psycopg_driver()`, which rewrites the scheme to `postgresql+psycopg://`. **Chosen over telling the developer to hand-edit the URL**, because that is a step to forget on every new environment, and the failure surfaces far from its cause. Also handles the legacy `postgres://` scheme.

4. Four tests failed because they were reading the real `.env`. `Settings(**values)` still loads the env file, so `apify_token` was populated from the developer's machine and the "later-milestone keys are optional" test failed. A test that passes for one person and fails for another is worse than no test.

Fixed with a `build()` helper passing `_env_file=None` — every config test is now hermetic.

5. mypy rejected a `# type: ignore[arg-type]`. The correct code was `call-arg`. Narrowed to `[call-arg,arg-type]` with a comment explaining that `_env_file` is a pydantic-settings init hook rather than a declared field.

The test database problem

`.env` still pointed at the **POC database** holding the 24 real captions. Today that was harmless — no models exist, so `create_all` created nothing. At P1 it would have started creating tables in there.

Safety rail added (`TEST_DATABASE_URL`):

- Unset → database-backed tests **skip**. A skipped test is a visible gap; a test that quietly rewrites real data is a disaster.
- Equal to `DATABASE_URL` → the suite **refuses to run**. That is never intentional.

How the databases were provisioned. Docker was available but heavy, and the daemon was not running. Rather than provisioning a second Railway service:

> A Postgres **server** hosts many **databases**, and Postgres databases cannot see each other's tables. Two extra databases on the existing instance give the same isolation at zero cost, with nothing new to run.

The Railway role turned out to be superuser with `CREATEDB`, so `scripts/create_databases.py` (idempotent, documented) created:

```
railway         ← POC data, 24 captions — untouched, referenced nowhere
sokolink        ← the application database
sokolink_test   ← the test database
```

Known limitation, accepted deliberately: this is *logical* isolation, not *resource* isolation — CPU, storage and connection limits are shared with the POC database. Fine for development. Before real sellers arrive, `sokolink` should move to its own Railway service; that is a dump and restore.

CI

`.github/workflows/ci.yml` runs ruff, ruff format, mypy, pytest, and `alembic upgrade head` on every PR, with a **real Postgres 17 service container** — because the DB-level constraints *are* the rails, and SQLite would silently accept

what Postgres rejects. No real secrets needed: external services are always mocked.

The migration step matters on its own: it proves the history applies to an empty database. A migration that cannot run from empty is a broken deploy.

Verification

Not "it should work" — actually run:

```
ruff          All checks passed
ruff format   14 files already formatted
mypy          Success — 13 source files (strict)
pytest        18 passed              (was 14 passed / 4 skipped before the DBs existed)
alembic       f33154a36521 (head), applied to sokolink

live uvicorn process, real HTTP, real round trip:
  GET /health        {"status":"ok","app":"SokoLink","env":"dev"}
  GET /health/ready  {"status":"ready","database":"up","detail":null}
```

The init migration mirrors TIKTOK's M0 — prove the pipeline runs end to end before there is a schema to migrate.

Still open

- **Railway deploy** — the last P0 item; needs the Railway dashboard.
- **Notion tracker** — the MCP connector worked once, then dropped and would not return across several restarts. Full rebuild spec written to `docs/NOTION_WORKSPACE.md` for a session that does have access.
- **Meta business verification** — developer account approved; verification pending. P2 onward waits on it. **Soko Intel does not**, which is why it can run in parallel.

2026-08-08 · P1 scope change and the data model

Scope: three ingestion paths, not one

The plan assumed one front door — sync a TikTok handle. That fails two-thirds of real sellers, so P1 now carries three:

PATH	SELLER DOES	SYSTEM DOES
Profile sync	enters `@handle`	Apify pulls the feed → AI drafts each video
Single link	pastes one TikTok URL	same pipeline, one item
Manual upload	uploads a photo	no scrape; AI drafts from the uploaded image

The third reuses the vision agent unchanged — it does not care whether an image came from a TikTok cover or a camera roll. Same code, new door.

Frontend decided: Jinja2 + HTMX + Tailwind

Not Next.js, despite TIKTOK having a working one. Keeping the whole application in one language and one deploy is the reason Python was chosen; HTMX covers inline editing, bulk actions, upload progress and live filtering without a JS framework or a second debugging context.

Two surfaces from one stack: the buyer storefront (mobile-first, minimal JS, also the SEO surface) and the seller dashboard (the working surface, draft review queue first).

The data model — rails in the database, not in services

`Seller`, `Product`, `ScrapeJob`, plus a `models/enums.py`.

Why the constraints live in Postgres rather than service code: a model, a future agent, a migration script and a hand-written query all have to obey the database. Only application code has to *remember* to call a validator.

Fifteen CHECK constraints landed. The ones that carry weight:

CONSTRAINT	PREVENTS
`published_requires_price`	the AI pushing an unpriced item live
`stock_non_negative`	overselling, at the storage layer
`price_positive`	a KSh 0 product — always a parse failure, never a giveaway
`price_plausible` (≤ 10,000,000)	the phone-number misparse reaching a storefront
`manual_has_no_video_id`	a manual product looking re-syncable
`published_needs_whatsapp`	a live shop with no way to contact the seller
`slug_format`	a URL a buyer cannot type
`failed_needs_error`	a failed scrape nobody can debug

Design decisions worth keeping:

- **Enums stored as strings with CHECK constraints**, not native PG enum types. Adding a value to a native enum needs a migration and an exclusive lock; adding one here is a code change. For categories, which will grow, that matters more than the bytes saved.
- `tiktok_video_id` **is nullable and unique.** Postgres permits multiple NULLs in a unique index, so manual products coexist with no partial index needed.
- `price_source` **records which cascade tier produced the price.** This is the feedback signal for the whole approach: if tier 3 rarely fires, the expensive path is not paying for itself. Without the column that is unanswerable.
- `raw_payload` **on ScrapeJob is kept deliberately** — it lets a changed parser be replayed against real data for free, and it is the evidence when a seller disputes an extraction.
- `Seller.tiktok_handle` **is nullable.** A manual-upload-only seller is legitimate; TikTok is an option, not a requirement.

Testing the rails

`tests/test_models.py` — 24 tests, and most assert that Postgres **refuses** something. A constraint nobody has watched fail is only a claim.

Autogenerate produced the migration; all 15 CHECK constraints carried through and were verified by reading the migration before applying it.

One fixture bug found by its own warnings

The rail tests emitted 13 × `SAWarning: transaction already deassociated from connection`. Cause: tests deliberately trigger `IntegrityError`, Postgres aborts the transaction, and the fixture then tried to roll back something already gone.

Fixed with an `is_active` guard. Isolation was never affected — each test gets a fresh connection either way — but a warning on every rail test trains you to ignore warnings, which is how the real one gets missed.

Known cost: the test suite is slow

42 tests take ~49 seconds, almost entirely network round trips to Railway. Fine now; it will hurt at 200 tests. The fix when it does is a local Postgres for tests — the shared-server choice bought zero setup at the price of latency.

Verification

```
ruff · format · mypy strict   clean
pytest                        42 passed, 0 warnings
alembic                       3bbf0cbc3f71 applied on top of f33154a36521
```

2026-08-08 · Spikes against a real seller — @zumamitumbabales

Test subject: a live Kenyan mitumba (second-hand clothing) seller. 22,000 followers, 1,453 videos, selling ladies' sandals in bulk.

Four spikes, each writing its raw output to `spikes/out/` so every later question is answerable offline and for free.

The cascade, measured on real data

TIER	SOURCE	YIELD	COST
1	Caption	**0/10**	~free
2	Cover image	**0/4**	cheap
3	**Video, audio + visual**	**3/3**	expensive

The cascade is not a nice-to-have — for this seller it is the only thing that works. Tiers 1 and 2 returned nothing at all. Had we built only caption parsing, this seller would have been uncatalogable.

Tier 2 is still worth keeping: it costs a fraction of tier 3 and other sellers do print prices on covers. Ordering by cost is the entire point.

Tier 2 failed *correctly*

Gemini identified the products confidently (0.90–0.95: "Mixed Ladies Sandals", "Assorted ladies sandals available for wholesale purchase in bulk lots") and returned `price_kes: null` on every one. It did **not** invent a plausible price. The guardrail — *never guess, a wrong price is worse than a missing one* — held under real conditions.

The finding that changed the data model

Tier 3 read the price from all three clips, and the `price_heard_as` field — added so a human could verify the model's reading — showed why that matters:

```
"@3000 30pairs"    "3000 for 30 pairs"    "3000 30pairs shown on screen"
```

This seller prices in bulk lots, not units. KES 3,000 buys *thirty pairs*. The account is literally named ZUMA MITUMBA **BALES**.

A storefront rendering "Mixed Ladies Sandals — KES 3,000" would make a buyer expect one pair. That is a trust-destroying error on the single field that must never mislead — and the `Product` model designed two hours earlier had no way to express it.

Added, driven entirely by the data:

- `unit_quantity` — 30
- `unit_label` — "pairs", in the seller's own vocabulary, not normalised
- `price_evidence` — the exact words the price was stated in, kept as the audit trail behind an AI-read number. It is what revealed bulk pricing at all.
- `price_display` — renders "KES 3,000 for 30 pairs", never the bare number
- Two constraints: a lot size must be positive, and quantity and label must be present together ("*KES 3,000 for 30*" *of what?*)

This is the whole argument for spiking before schema, demonstrated on ourselves. The model was reasonable, well-documented, fully tested — and wrong about a real seller in a way no amount of thinking would have surfaced.

Other findings worth keeping

- **No TikTok transcriptions.** `transcriptionLink` and `subtitleLinks` were null on 10/10, matching TIKTOK's spike 00. Tier 3 must send actual video; there is no cheap text shortcut.
- **Cover URLs are signed and expire.** We must store our own copies, as designed.
- **The bio carries phone numbers.** `signature` held "*...contact us 0105515839/0754234636/0762508007*" — so `whatsapp_number` can be pre-filled at onboarding and merely confirmed. One less field to type.
- **Clips are short** — 9–24s, mean 13.8s. Tier 3 is affordable per item, but must stay cached once-ever per video id.
- **1,453 videos on this profile.** A full import is ~$0.44 in Apify alone, and most of it is stale stock. Initial sync must be capped at the recent N, not the whole history.
- **Hashtags carry category, never price** — `#sandalsforwomen`, `#zarasandals`, `#kidscrocs`. Useful as a hint to the vision model, worthless as a price source.
- **Engagement baseline is thin at 10 posts** (mean 563 views, no 3× outliers). Soko Intel's outlier detection needs a fuller feed to mean anything.

Three failures along the way

1. `gemini-2.5-flash` is retired. 404 "*no longer available to new users*". Switched to `gemini-3.6-flash`, which is also what TIKTOK validated on for Sheng/Swahili. Model ids in config are a maintenance surface, not a constant.

2. Apify's downloaded videos are not public. `mediaUrls` points into Apify's key-value store and 403s without the API token. My spike fetched a 214-byte JSON error and posted it to Gemini **as a video** — a paid call spent on garbage, with no complaint. Fixed by adding the token, and by refusing to send anything whose content-type is not video or that is implausibly small. Exactly the silent failure our own standard bans, committed by me.

3. `create_all` does not migrate. The session fixture created missing tables but never altered existing ones, so adding three columns left the test database stale and produced 25 failures with a baffling "*column unit_quantity*

does not exist". Now drops and recreates, which cannot drift. Safe because `TEST_DATABASE_URL` is refused if it equals the app database.

Verification

```
ruff · format · mypy strict   clean
pytest                        50 passed
alembic                       588f6c5577d0 (head)
```

2026-08-08 · Ingestion and the cascade, in production code

The spikes are now the specification. Three layers, each with one job.

`schemas/tiktok.py` — the validation border

Apify's response is a third-party contract we do not control. Validating once, at the edge, means a shape change fails loudly with a readable error instead of surfacing three layers later as a `None`.

The models are **tolerant of extra fields and strict about the ones we depend on**. Apify sends 28 top-level keys; we need nine. A field they add can never break us; one they remove that we rely on does — which is the right way round.

Field names come from the real payload, not from documentation.

`services/scraper.py` — the adapter

Callers depend on `ScraperEngine`, never on Apify. Swapping the engine is a change to one file. TIKTOK proved the value of this seam by swapping vision providers three times through an equivalent one.

`download_media()` refuses anything whose content type is not what was asked for. That guard exists because a spike fetched a 214-byte JSON error from Apify's key-value store and passed it to Gemini **as a video** — a paid call spent on nothing, silently.

`DEFAULT_PROFILE_LIMIT = 30`, because the spike account had 1,453 videos. A full import is ~$0.44 of Apify for one seller, mostly stale stock.

`agent/draft.py` — the provider seam

The only file that knows which vision provider we use. Constrained decoding against `ProductDraft`, so the output can only be a valid instance.

One guard worth naming: **constrained decoding guarantees the SHAPE, never the SENSE.** A model can still return a phone number in a price field. The agent rejects an implausible price itself, so the failure names the draft rather than arriving as an IntegrityError from three layers down.

`services/drafting.py` — where the economics live

The agent knows how to ask a model; this knows how much we are willing to pay to find out. Conflating them is how a cost control ends up buried in a prompt.

- Tier 1 (caption) is **skipped as a price source on purpose** — 0/10 measured, and every caption already reaches the model as hashtag context. A separate text call would spend money re-reading what tier 2 sees anyway.
- A confident tier-2 price **stops** the cascade.

- A failed tier does **not** abort it — the next tier may still rescue the item — but the failure is recorded, never swallowed.
- `allow_video_tier=False` exists for bulk imports and over-quota sellers.
- `tiers_attempted` is recorded so we can ask later whether the expensive tier earns its cost. Unrecorded, that question is unanswerable.

Testing what actually matters here

The cascade tests protect **cost**, not just correctness. A regression that quietly escalated every item would pass a naive correctness test and arrive as a bill — so the tests assert which tiers ran, not only what came out.

Scraper fixtures are built from the real captured payload. A test that passes against a payload we imagined proves nothing about the one the provider sends.

Verification

```
ruff · format · mypy strict   clean
pytest                        92 passed
```

2026-08-09 · Multi-platform, and logins

The schema was TikTok-shaped

Instagram, Facebook and later Jumia entered the plan. The model could not hold them: `Seller.tiktok_handle` is one platform and one handle, and `Product.tiktok_video_id` cannot express "this came from IG post X".

Generalised now, deliberately, with zero production rows. A migration nobody notices today; backfilling fifty live seller catalogues later.

BEFORE	AFTER
`Seller.tiktok_handle`	`SocialAccount` rows — a seller has many
`Product.source` (conflated)	`platform` + `ingest_method` — two dimensions
`Product.tiktok_video_id`	`platform_post_id`, unique **per platform**

Why a table rather than more columns: a column per platform means a migration for every platform added, and nowhere to record per-account state like "when did we last sync *this* one". A row per connection makes adding Jumia a data change rather than a schema change.

Why two provenance dimensions: platform says *where it came from*, ingest_method says *how it got here* — and only the second decides re-sync ownership. `IngestMethod.is_sync_owned` is the guard: a feed sync owns only what it created.

Two new constraints fell out of the split:

- `manual_iff_upload` — a "tiktok upload" is incoherent; the dimensions must agree.
- `uq_products_platform_post` — uniqueness is **per platform**, because two platforms can legitimately mint the same numeric id and a global unique would reject the second.

Only TikTok has an engine. Instagram and Facebook are declared so the schema, URLs and UI never change to admit them; `ScraperEngine` is already a protocol, so a new engine slots in beside `ApifyEngine` without callers changing.

Auth — email + password now, phone later

Phone + OTP is the better fit (the number is already the seller's identity, and it is what M-Pesa charges) but it needs an SMS provider we do not have. `Account.phone` exists from the start so that migration is additive.

`app/security.py` owns hashing and signing; nothing else touches a password.

Anti-enumeration is the whole point of `services/accounts.py`. Three measures, all load-bearing and all tested:

1. **One message for every failure** — unknown email, wrong password and deactivated account are indistinguishable. "Your account is disabled" would itself confirm the address exists.
2. **A timing equaliser.** When no account is found we still verify against `DUMMY_HASH`, so a missing email costs the same Argon2 work as a wrong password. Without it, response time is an oracle measurable over the network — and there is a test asserting the unknown-email path is not an order of magnitude faster.
3. **No early return** before that comparison.

Signup *can* say "email already registered" — a stranger reaching that message already claimed the address, and hiding it would leave them stuck. Login never can. The asymmetry is deliberate.

Other decisions worth keeping:

- **Signup creates the Seller too**, in one transaction. A login with no shop is a dead end.
- **Slug collisions append `-2`** rather than failing. A taken shop name must not block signup; the slug is not permanent until publish.
- **`RESERVED_SLUGS`** stops a seller taking `/login` or impersonating the platform — with a test asserting every entry is lowercase and self-slugifying, so a typo cannot silently disable the guard.
- **Password rule is length only.** Character-class rules push people toward `Passw0rd!`; a long phrase is stronger and far easier on a phone keyboard.
- **`Account.__repr__` omits the email**, because repr lands in logs.

Verification

```
ruff · format · mypy strict   clean
pytest                        145 passed
alembic                       d6cdd06e2776 (head)
```

2026-08-09 → 15 · Ownership, ingestion, and the buyer storefront

Three milestones logged together, because they are one argument: a handle someone typed is worthless until it is proven, a proven account is worthless until it is synced, and a synced catalogue is worthless until a buyer can see it.

The security change that reshaped the schema

Fredrick's recommendation, after reading the codebase: "*if you cannot verify we should not accept the account at all. The account should only be accepted if you can verify.*"

The obvious implementation is a nullable `verified_at` and a filter everywhere. That is a rule someone has to remember, on every query, forever — and the first query that forgets it publishes a storefront pointing at the wrong WhatsApp number.

What was built instead: an unverified connection is unrepresentable.

```
AccountClaim              SocialAccount
unproven, expires         verified_at           NOT NULL
grants nothing            verification_method   NOT NULL
```

Two tables, not one table with a flag. `SocialAccount.verified_at` being NOT NULL means the database cannot hold an unproven connection, so nothing downstream filters for verified accounts — there is no other kind. `services/verification.py` has exactly one function that creates a `SocialAccount` (`_connect`), and every path to it goes through proof.

The attack this defeats is not hypothetical: a stranger claims another seller's handle, we scrape her videos and her photos, and publish a shop pointing at *their* WhatsApp number. The buyer sees nothing wrong. That is sales diversion, and one incident reaching Kenyan seller groups would be very hard to come back from.

Why bio-code and not OAuth. OAuth is better — one tap, the platform vouches, nothing to retype. It is unavailable until TikTok and Meta approve our app, on their clock, and reading a seller's posts needs a second scope with its own approval. Sellers arrive before either lands. `complete_via_oauth` is written and tested, so switching is a call-site change rather than a rewrite.

What bio-code does not prove: someone with temporary access to the account could pass it. It defeats the realistic attack — a stranger typing a handle they have never touched — and that is the bar it is built to clear. Stated in the module docstring so nobody later mistakes it for something stronger.

Small decisions inside it that came from picturing a phone keyboard:

- `_CODE_ALPHABET` **drops** `0 O 1 I L`**.** The seller retypes this into a bio on a phone. Ambiguous characters are exactly where that goes wrong.
- **A** `soko-` **prefix**, so the string is identifiable in a bio full of hashtags and phone numbers. *Still* `soko-` *after the rename — see Still open, below.*
- **24-hour claim lifetime, capped attempts.** Both surfaced to the seller honestly; a silent limit reads as a broken product.

Ingestion — three money guards

`services/ingestion.py` is where the unit economics are enforced:

1. **Once per day per account.** Apify bills per post. Syncing on every dashboard load makes the economics fail *exactly when the product succeeds*. Checked **before** the paid call, with a test asserting that ordering.
2. **Skip posts already drafted.** A re-sync updates metrics but never re-runs the AI on a post already processed — that work is done and paid for.
3. **The video tier is opt-in per sync** (`allow_video_tier=False`). A bulk import that watched every clip would be ruinous. The seller escalates deliberately, per item.

A regression in any of these arrives as a bill, not as a failing assertion — unless there is an assertion. There are nineteen.

The re-sync rail. A sync may only touch products it created (`ingest_method == profile_sync`). Expressed as a query scope rather than a rule to remember: hand-uploaded stock is simply invisible to `_existing_by_post_id`. A seller who uploads stock, syncs their feed and watches it vanish does not come back.

A failed sync does not start the cooldown. Otherwise one Apify hiccup locks the seller out for a day. The failed job is still recorded before raising, because a failure the seller cannot see is indistinguishable from "you have no videos" — which sends them away thinking we are broken.

Media — their URLs expire, so we keep our own copies

Spike 02 flagged that TikTok cover URLs are signed and carry an `x-expires` timestamp. The demo storefront then proved it, rendering ten broken images a week after the scrape.

`services/media.py` stores a copy keyed by post id, and products store a **relative** path — so moving to object storage later changes one constant.

A bug found while writing this milestone's tests: `store_cover` promised in its own docstring never to let an image failure cost a product, but caught only `ScraperError`. A timeout, a full disk, or an engine that had not implemented the method took the whole sync down with it. The catch is now deliberately broad, and the reason is written above it — a function whose entire contract is "never let this cost us a product" cannot enumerate its failures in advance.

The buyer storefront

`services/storefront.py` + `api/storefront.py` + two templates. Server-rendered, no client framework, lazy images below the fold, because it opens inside the WhatsApp in-app browser on a cheap Android over expensive data.

- **Price is never a bare number.** `price_display` carries the units, because "KES 3,000" and "KES 3,000 for 30 pairs" mean very different things to someone deciding whether to send money.
- **Unpublished shops 404 exactly like missing ones.** An unpublished shop is full of half-parsed drafts nobody should see, and distinguishing the two cases leaks which slugs are taken.
- **No phone number in a URL path.** The WhatsApp handoff builds `wa.me` links at render time from the seller record.

Verification

```
ruff · format · mypy strict   clean
pytest                        199 passed
```

2026-08-16 · The web layer — a design system, and the login wall

Context: the product changed shape. Fredrick's direction — lead with the AI content and growth engine, not the store. *"In order for the store to be successful we need a way for people to know that we have a store. The way to do that is to make social tools for creators."* And: a proper web platform **before** WhatsApp, not after it. Nothing in this phase depends on Meta.

`docs/PHASE_0.md` is the seven-step plan that came out of that. This entry is steps 1–3.

Also: **SokoLink became Biashara Mall.** `sokolink.com` was taken; `biasharamall.com` is live on a landing page.

Tailwind was dropped, deliberately

The plan said Jinja2 + HTMX + Tailwind. Tailwind offers two options and both were wrong for this week:

OPTION	WHY NOT
Build step (Node, or the standalone binary)	One more thing to break in the Railway deploy, and deploying early is the plan
CDN runtime	Ships ~3MB of JS and generates the CSS **in the browser**

The users are on cheap Android handsets over expensive, patchy data. Every kilobyte is a creator who leaves before the page paints. `app/static/css/app.css` is **540 lines, 20.7KB uncompressed (~4KB gzipped)**, and the deploy stays `pip install` and nothing else.

It is reversible. The components map closely onto utility classes, so adopting Tailwind later is a rewrite of one file.

Every colour lives in one `:root` block. Rebranding to match the landing page is a single edit, not a hunt through templates. The palette is green because this market reads green as growth and money, and WhatsApp already trained the commerce association — but that choice is deliberately one edit deep.

Two base templates, on purpose

```
base.html       the buyer's storefront   — a stranger, one visit, cheapest data
app_base.html   the creator's workspace  — signed up, weekly, tables and charts
```

They have nothing in common but the company name. Keeping them separate means a change to the dashboard cannot slow down the storefront, which is the page that has to be fastest. The storefront keeps its Tailwind CDN until it is next touched — rewriting a working page to satisfy a consistency rule is not a good trade with buyers already on it.

The session layer — `app/dependencies.py`

`app/security.py` already minted and read signed tokens. What was missing was the HTTP wrapper, and four things that have to agree:

- `HttpOnly` — so an XSS anywhere on the site is not account theft
- `SameSite=Lax` — also our CSRF defence, since every form here is same-site
- `Secure` **follows the environment**. Hardcoded on, and localhost silently drops the cookie: "login works but nothing is remembered", which is a miserable afternoon
- `max-age` matching the token's own expiry

Written once in `set_session_cookie`, not per route.

A redirect, not a 401. Everything above this is a browser. A 401 leaves a seller staring at `{"detail":"Not authenticated"}` with no way forward. The `LoginRequired` exception carries `?next=` and is handled application-wide in `main.py`, so no page behind the wall can forget to do it.

303, not 302 — it guarantees the browser re-issues as GET, so a POST that hit an expired session is not replayed after login.

`?next=` is attacker-controlled

The one genuinely dangerous thing in this layer. Reflecting it unchecked turns our login page into a phishing hop: a real `biasharamall.com` login URL that bounces to someone's clone with the credentials still fresh in mind.

Only a **single** leading `/` is accepted. `//evil.example` is a protocol-relative URL and is the easy one to miss — it has its own test. Sanitised on the way *in* as well as on the way out, or the hidden form field carries the hostile value back to us on the next submit.

Anti-enumeration had to survive the round trip

`services/accounts.py` refuses to say whether an email exists. A template rendering a friendlier message, or a status code that differs by cause, hands that straight back. `api/auth.py` renders `str(exc)` rather than a message of its own — one place decides what a failed login says, and it is the service. Tested at the HTTP layer: same status, same string, for unknown email and wrong password.

Two smaller decisions from the same instinct:

- **The typed email survives a failure; the password never does.** Retyping everything after one mistake is how you lose a signup. A re-rendered password lands in proxy logs, browser caches and the back button.
- **No email confirmation before first use.** Verification earns its place when there is something worth protecting behind it. Today it is a wall in front of an empty dashboard.

Two things found while wiring it up

`api/storefront.py` was building its own Jinja environment. The moment a second router rendered HTML that meant two environments, and a `| media` filter existing in only one of them — a dashboard image would render a raw database path and nobody would know why. Extracted to `app/templating.py`; a filter registered there cannot be forgotten, because no template has to apply it.

`RESERVED_SLUGS` was missing `analytics` and `accounts`. Both are routes introduced this session. Not a security hole — routes register before the storefront's `/{slug}` — but a seller claiming either would have got a silently unreachable shop.

The empty state is the product's first impression

Nearly every creator who ever signs up sees the dashboard with nothing on it, and what it says decides whether they take the one action that makes the product work. So it holds one sentence, one reason and one button — plus the line that answers the objection before it is raised:

> No password needed — you paste a short code into your bio.

Fear of a password grab is the most common reason a creator abandons this screen.

What was built

FILE	LINES	
`app/static/css/app.css`	540	The design system. One `:root` block owns every colour
`app/dependencies.py`	180	Session layer — cookies, `current_account`, the login wall
`app/api/auth.py`	214	signup / login / logout
`app/api/dashboard.py`	72	The shell and its empty state
`app/templating.py`	42	One Jinja environment, shared
`app/templates/app_base.html`	74	Workspace shell
`app/templates/auth/login.html`	92	
`app/templates/auth/signup.html`	95	
`app/templates/app/dashboard.html`	85	
`tests/test_web_auth.py`	268	20 tests: pages, cookies, redirects, enumeration

Verification

```
ruff · format · mypy strict   clean
pytest                        219 passed
alembic                       0f28636c2759 (head)
```

No migration — this session added no columns.

Still open

1. **Brand colours.** The palette was chosen, not matched: `biasharamall.com` serves its CSS in a form `WebFetch` strips, so the hex codes could not be read programmatically. One edit to `:root` when they arrive.
2. **The verification code prefix is still `soko-`.** Changing it to `bm-` is a one-line change that invalidates any code already sitting in a bio. Nobody has one yet — this is the free moment, and it closes at Step 4.
3. **No Railway deploy config.** No `railway.json` and no `Procfile`, so Railway has no start command. It needs `uvicorn app.main:app --host 0.0.0.0 --port $PORT`. Everything above is now worth deploying.
4. **Eight commits unpushed.**

2026-08-17 · The auth screens, rebuilt to the brand mockup

Fredrick supplied a mockup of the signup screen from `biasharamall.com`. The first build was a centred card; this is a split screen with the product case beside the form. Rebuilt to match.

What the mockup settled that guesswork had not

- **The brand green.** The palette had been guessed as emerald (`#059669`) because the landing page's CSS could not be read programmatically. It is a true green — the scale is now `#22c55e / #16a34a / #15803d`. One `:root` edit, as designed.

- **The naming.** The mockup reads **Biashara Intel** and **AI Content Brain**, not Soko Intel. The pillar names in `CLAUDE.md` are now out of step with the brand — flagged, not silently changed.
- **The onboarding shape.** *Create account → Make content → Review & publish.* That is not the flow `PHASE_0.md` planned (connect TikTok → sync → analytics). The step meter is built and honest about being step 1 of 3; **steps 2 and 3 do not exist yet**, and which flow wins is an open decision.

Why a marketing pane next to a form

Most people reach `/signup` from an ad, a WhatsApp forward or a TikTok bio, having read no landing page. A bare form asks them to commit to something unexplained. The left pane is the explanation, placed where the hesitation is.

On mobile the form comes first. `order` flips the panes: a phone shows one at a time, and someone who tapped "Sign up" has already decided. Making them scroll past a pitch taxes the people who need convincing least.

Two enhancements, neither of them a dependency

The live workspace-slug preview. A creator who watches their web address form as they type chooses a better name than one told, after the fact, what we picked. It needs client-side slugification — which means **one rule with two implementations**, in `services/accounts.py` and `static/js/auth.js`.

That drift is not cosmetic: a creator sees one address and gets another, at exactly the moment they are deciding whether to trust us. So `tests/test_slug_parity.py` extracts `slugify()` **from the shipped JS file** — not a copy, which would drift alongside it — and runs both over sixteen inputs chosen for disagreement: accents, emoji, Cyrillic, truncation landing on a separator, runs of punctuation. Skipped rather than failed when Node is absent, since CI is Python-only.

The preview stays a preview: the server appends `-2`, `-3` on collision and alone knows what is taken.

A password reveal toggle, because the alternative on a phone keyboard is a confirm-password field — one more thing to type, and a common place a signup is abandoned.

Both are progressive enhancement. Both pages work with JavaScript blocked, failed or still downloading, which is the normal case on a patchy connection.

The logo read as a padlock

First attempt: a uniformly rounded bag body under a wide handle arc. At 30px it was unmistakably a padlock — the last thing a signup page should accidentally say. Fixed by narrowing the handle relative to the bag and squaring the bag's top corners while keeping the base rounded, so it reads as a tote.

Caught by screenshotting the rendered page rather than by reading the markup. Worth remembering: SVG at small sizes cannot be reviewed as source.

The mark is an approximation of the real one. **Ask for the actual asset.**

Verification

```
ruff · format · mypy strict   clean
pytest                        220 passed
app.css                       28.0KB uncompressed
```

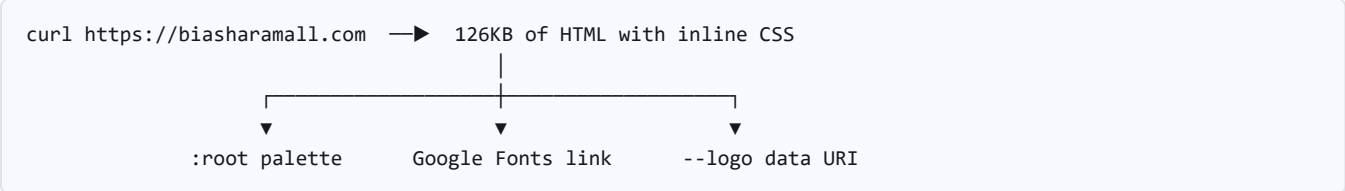
Screenshotted at 1280px and at 520px before being called done.

Still open, from this session

1. **The real logo file**, to replace the hand-drawn approximation in `templates/partials/logo.html`.
2. **The wordmark is "Biasharamall"**, one word, matching the mockup — while `config.app_name` and every document say "Biashara Mall". One of them is wrong.
3. **Soko Intel vs Biashara Intel**, and whether Soko Commerce / Soko AI rename with it.
4. **Which onboarding flow is real** — the mockup's three steps, or Phase 0's connect-then-analyse. The step meter currently promises the former.
5. **The typeface.** The mockup uses a geometric sans; this uses the system stack, because a web font is a render-blocking download on a slow connection. Revisit if the brand look matters more than the first paint.

2026-08-18 · The real brand, read from the source

The hand-drawn logo was wrong and Fredrick said so. The fix turned out to be much larger than a logo: **the landing page's stylesheet is fetchable, and it contains every token we had been guessing at.**



No login, no browser automation, no account access — the page is public and serves its own design system.

The palette had been guessed twice, and corrected in the wrong direction

ATTEMPT	VALUE	
First build	emerald `#059669`	guessed from memory — **right by luck**
Second build	grass `#16a34a`	read off a screenshot — **wrong**
Now	emerald `#059669`	read from the stylesheet

The screenshot looked like a truer green than emerald, so the palette was "corrected" away from the correct answer. **A rendered screenshot is not a colour source.** Anti-aliasing, the display profile and PNG quantisation all sit between the value and the pixel.

What the real `:root` gave us, none of which was guessable:

- **Green-tinted neutrals**, not grey ones — `--ink:#0a1712`, `--muted:#5b6b63`, `--line:#e7ebe8`, `--canvas:#f7faf8`. Subtle, and the reason the brand's whites look warm beside the emerald rather than clinical.
- **Green-tinted shadows** — `rgba(4,54,31,…)`, never black. A neutral drop-shadow under an emerald card reads muddy. This one detail did more for the match than the hue did.
- **A softer radius scale** — 10 / 16 / 24 / 32 against our 6 / 10 / 16.
- `--maxw:1180px`, and an easing curve.

The typeface decision reversed

The brand pairs **Sora** (display) with **Inter** (text), from Google Fonts. This file had argued for the system stack on the grounds that a web font is a render-blocking download on a slow connection.

Two things changed the calculus:

1. The brand actually has a typeface, and "close enough" typography is the most visible way a product looks unfinished.
2. A creator arriving from `biasharamall.com` has **already downloaded both**. For the common path it is a cache hit, not a download.

Loaded from the identical URL the landing page uses, with `preconnect` and `display=swap`, so text paints immediately in the fallback and swaps when the font lands. The worst case is a swap, never a blank screen.

Sora is on headings only. It is a display face and gets tiring at paragraph sizes — which is exactly how the brand uses it.

The logo is now the brand's own file

It ships on the landing page as a base64 PNG in a `--logo` custom property. Decoded to `static/img/logo.png` (120×120, 16.7KB): a green bag with a yellow map pin, forming a B.

The hand-drawn SVG that replaced it twice is gone. Do not reintroduce a drawn version — if the mark changes, replace the file.

What the real font immediately broke

Sora is wider than the system stack. Two things that fitted before did not:

- The "About 1 minute" pill wrapped onto two lines — a pill that wraps stops being a pill. Fixed with `white-space: nowrap`.
- That wrap then stole a line from the heading beside it. `.auth-card-head h1` is now sized to sit on one line at the narrowest column the split grid allows (400px).

Neither was visible in the markup. Both were caught by screenshotting the rendered page — the same way the padlock logo was caught. **Typography and small-format SVG cannot be reviewed as source.**

Verification

```
ruff · format · mypy strict   clean
pytest                        220 passed
app.css                       30.5KB uncompressed
logo.png                      16.7KB
```

Screenshotted at 1280px and 520px.

Still open

1. **Wordmark spelling.** The brand renders "Biasharamall", one word; `config.app_name` and every document say "Biashara Mall". Unresolved.
2. **Pillar names.** The landing nav reads **BiasharaIntel**; `CLAUDE.md` declares Soko Intel / Soko Commerce / Soko AI. The brand has moved and the docs have not.
3. **Which onboarding is real** — the mockup's *Create account → Make content → Review & publish*, or Phase 0's *connect → sync → analytics*. The step meter currently promises the former, and steps 2 and 3 do not exist.
4. **The storefront still runs on the Tailwind CDN** and now looks like a different product from the workspace. It should adopt these tokens next time it is touched.